

Job Description: Senior AI Engineer — Founding Team

Company: Redrob AI (Series A AI-native talent intelligence platform)

Location: Pune/Noida, India (Hybrid — flexible cadence) | Open to relocation candidates from Tier-1 Indian cities

Employment Type: Full-time

Experience Required: 5–9 years (see "what we mean by this" below)

Let's be honest about this role

We're going to write this JD differently from most. We're a Series A company that just raised our round and we're building a new AI Engineering org from scratch. This is the kind of role where the JD changes every six months because the company changes every six months. So instead of pretending we have a fixed checklist, we're going to tell you what we actually need and what we've gotten wrong before.

If you've spent your career at Google or Meta and you want a well-scoped role with a defined ladder, this isn't it.

If you've spent your career bouncing between early-stage startups and you want to "just code" without having to think about product or recruiter workflows or eval frameworks, this also isn't it.

We need someone who is simultaneously comfortable with two things that sound contradictory:

1. Deep technical depth in modern ML systems — embeddings, retrieval, ranking, LLMs, fine-tuning.
2. Scrappy product-engineering attitude — willing to ship a working ranker in a week even if the underlying ML is "obviously suboptimal," because we need to learn from real users before we know what to actually optimize for.

These are not contradictory in real life. They feel contradictory because of how engineering culture sorted itself into "researcher" vs "shipper" archetypes. We need both modes available in the same person, and we'd rather you tilt slightly toward shipper than toward researcher.

What you'd actually be doing

The high-level mandate: own the intelligence layer of Redrob's product. That means the ranking, retrieval, and matching systems that decide what recruiters see when they search for candidates and what candidates see when they search for roles.

In practical terms, your first 90 days will probably look like:

- Weeks 1-3: Audit what we currently have (it's mostly BM25 + rule-based scoring, working but not great). Identify the 3-4 highest-leverage things to fix.
- Weeks 4-8: Ship a v2 ranking system that demonstrably improves recruiter-engagement metrics. This will involve embeddings, hybrid retrieval, and probably some LLM-based re-ranking, but the architecture is your call.
- Weeks 9-12: Set up the evaluation infrastructure — offline benchmarks, online A/B testing, recruiter-feedback loops — so we can keep improving without flying blind.

Beyond that, you'll be driving the long-term architecture of how we do candidate-JD matching at scale, mentoring the next round of hires (we're growing the team from 4 to 12 engineers in the next year), and working closely with our recruiter-experience PM on what to build.

What we mean by "5-9 years"

This is a range, not a requirement. Some people hit "senior engineer" judgment at 4 years; some never hit it after 15. We've used 5-9 because it's roughly where people we've hired into this kind of role have landed, but we'll seriously consider candidates outside the band if other signals are strong.

That said, here are the disqualifiers we actually apply:

- If you've spent your career in pure research environments (academic labs, research-only roles) without any production deployment — we will not move forward. We are explicit about this. We've tried it twice and it didn't work for either side.
 - If your "AI experience" consists primarily of recent (under 12 months) projects using LangChain to call OpenAI — we will probably not move forward, unless you can demonstrate substantial pre-LLM-era ML production experience. We're looking for people who understood retrieval and ranking *before* it became fashionable.
 - If you are a senior engineer who hasn't written production code in the last 18 months because you've moved into "architecture" or "tech lead" roles — we will probably not move forward. This role writes code.
-

The skills inventory (please read carefully)

Most JDs list 20 skills and you're supposed to have all of them. We're going to do this differently.

Things you absolutely need

- Production experience with embeddings-based retrieval systems (sentence-transformers, OpenAI embeddings, BGE, E5, or similar) deployed to real users. We don't care which model — we care that you've handled embedding drift, index refresh, retrieval-quality regression in production.
- Production experience with vector databases or hybrid search infrastructure — Pinecone, Weaviate, Qdrant, Milvus, OpenSearch, Elasticsearch, FAISS, or something similar. Again, the specific tech doesn't matter; the operational experience does.
- Strong Python. Yes really, we care about code quality.
- Hands-on experience designing evaluation frameworks for ranking systems — NDCG, MRR, MAP, offline-to-online correlation, A/B test interpretation. If you've never thought about how to evaluate a ranking system rigorously, this role will be very painful.

Things we'd like you to have but won't reject you for

- LLM fine-tuning experience (LoRA, QLoRA, PEFT)
- Experience with learning-to-rank models (XGBoost-based or neural)
- Prior exposure to HR-tech, recruiting tech, or marketplace products
- Background in distributed systems or large-scale inference optimization
- Open-source contributions in the AI/ML space

Things we explicitly do NOT want

This is the section most JDs skip but we think it's the most important:

- Title-chasers. If your career trajectory shows you optimizing for "Senior" → "Staff" → "Principal" titles by switching companies every 1.5 years, we're not a fit. We need someone who plans to be here for 3+ years.
 - Framework enthusiasts. If your GitHub is full of LangChain tutorials and your blog posts are "How I used [hot framework] to build [demo]" — that's fine but it's not what we need. We need people who think about systems, not frameworks.
 - People who have only worked at consulting firms (TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini, etc.) in their entire career. We've had bad fit experiences in both directions. If you're currently at one of these companies but have prior product-company experience, that's fine.
 - People whose primary expertise is computer vision, speech, or robotics without significant NLP/IR exposure. We respect your work but you'd be re-learning fundamentals here.
 - People whose work has been entirely on closed-source proprietary systems for 5+ years without external validation (papers, talks, open-source). We need to see how you think, not just trust that you can think.
-

On location, comp, and logistics

- Location: Pune/Noida-preferred but flexible. We have offices in Noida and Pune(mostly used Tue/Thu). We don't require any specific number of in-office days but we expect quarterly travel for offsites. Candidates in Hyderabad, Pune, Mumbai, Delhi NCR welcome to apply. Outside India: case-by-case, but we don't sponsor work visas.
 - Notice period: We'd love sub-30-day notice. We can buy out up to 30 days. 30+ day notice candidates are still in scope but the bar gets higher.
-

The vibe check

We genuinely believe culture-fit matters more at this stage than skills-fit. Skills are teachable; the rest mostly isn't.

We work async-first and write a lot. If you find writing painful, you'll find this role painful.

We disagree openly and decide quickly. If you find that style abrasive, you'll find this role abrasive.

We move fast and break things, with the caveat that "things" are usually our internal assumptions, not user-facing systems. If you need a stable, mature codebase to be productive, you'll find this role unstable.

How to read between the lines

The "ideal candidate" we're imagining is roughly:

- 6-8 years total experience, of which 4-5 are in applied ML/AI roles at product companies (not pure services).
- Has shipped at least one end-to-end ranking, search, or recommendation system to real users at meaningful scale.
- Has strong opinions about retrieval (hybrid vs dense), evaluation (offline vs online), and LLM integration (when to fine-tune vs prompt) — and can defend them with reference to systems they actually built.
- Located in or willing to relocate to Noida or Pune.
- Active on Redrob platform (or has clear signal of being in the job market) so we can actually talk to them.

We are aware this is a narrow profile. We're not expecting to find many matches in a 100K candidate pool. We're explicitly OK with that — we'd rather see 10 great matches than 1000 maybes.

Final note for the participants of the Redrob hackathon

If you're reading this in the context of the Intelligent Candidate Discovery & Ranking Challenge:

The "right answer" to this JD is not "find candidates whose skills section contains the most AI keywords." That's a trap we've explicitly built into the dataset.

The right answer involves reasoning about the gap between what the JD says and what the JD means. A Tier 5 candidate may not use the words "RAG" or "Pinecone" in their profile, but if their career history shows they built a recommendation system at a product company, they're a fit. A candidate who has all the AI keywords listed as skills but whose title is "Marketing Manager" is not a fit, no matter how perfect their skill list looks.

Your ranking system should also weigh behavioral signals — a perfect-on-paper candidate who hasn't logged in for 6 months and has a 5% recruiter response rate is, for hiring purposes, not actually available. Down-weight them appropriately.

Good luck.

Redrob Behavioral Signals — Reference

This document explains the 23 behavioral signals embedded in each candidate's `redrob_signals` object, how they relate to candidate quality, and how they're constructed in the synthetic dataset.

What are Redrob signals?

In a real recruiting platform, candidates generate observable behavior beyond what they list in their profile:

- Do they actually respond to recruiter messages?
- Have they logged in recently?
- Did they complete the assessments they started?
- Are recruiters saving their profile?
- Have they completed previous interview cycles?

These behavioral signals are often more predictive of whether a candidate can actually be hired than their static profile. A perfect-on-paper candidate who hasn't logged in for 6 months and has a 5% response rate is, for hiring purposes, not actually available.

This dataset includes these signals so that ranking systems can incorporate them as a multiplier or modifier on top of skill-match scoring.

The 23 signals

#	Signal	Range / type	What it measures
1	<code>profile_completeness_score</code>	0-100	How much of the profile they've filled in
2	<code>signup_date</code>	date string	When they signed up on Redrob
3	<code>last_active_date</code>	date string	When they last logged in
4	<code>open_to_work_flag</code>	bool	Have they marked themselves available
5	<code>profile_views_received_30d</code>	integer ≥ 0	How often their profile has been viewed by

			recruiters in last 30 days
6	applications_submitted_30d	integer >= 0	How many roles they've applied to recently
7	recruiter_response_rate	0.0-1.0	What fraction of recruiter messages they reply to
8	avg_response_time_hours	number >= 0	Median time to respond to a recruiter message
9	skill_assessment_scores	dict[str, 0-100]	Per-skill Redrob assessment scores
10	connection_count	integer >= 0	Number of Redrob connections
11	endorsements_received	integer >= 0	Total skill endorsements received
12	notice_period_days	0-180	Their stated notice period
13	expected_salary_range_inr_lpa.min / .max	number >= 0	Salary expectations in INR lakhs per annum
14	preferred_work_mode	onsite/hybrid/remote/flexible	Their stated work-mode preference
15	willing_to_relocate	bool	Will they relocate if needed
16	github_activity_score	-1 to 100	GitHub commits/contributions score (-1 if no GitHub linked)
17	search_appearance_30d	integer >= 0	How often they show up in recruiter searches
18	saved_by_recruiters_30d	integer >= 0	How many recruiters bookmarked them in last 30 days

19	interview_completion_rate	0.0-1.0	What fraction of interviews they've actually attended
20	offer_acceptance_rate	-1 to 1.0	What fraction of offers they accepted (-1 if no prior offers)
21	verified_email	bool	Whether their email address is verified
22	verified_phone	bool	Whether their phone number is verified
23	linkedin_connected	bool	Whether their LinkedIn account is connected

Submission Specification — Redrob Hackathon v4

Read this carefully before submitting. Submissions that don't match this spec will be auto-rejected by the validator without scoring.

1. What you're submitting

A CSV file ranking the top 100 candidates from candidates.jsonl for the released job description.

Rank 1 is the best fit; rank 100 is the 100th best fit.

You do *not* rank candidates 101 onward — only the top 100.

2. File format

Filename

Your team's registered participant ID, with .csv extension. For example: team_xxx.csv.

Encoding

UTF-8.

Required columns (in this order)

```
candidate_id,rank,score,reasoning
```

Column	Type	Required?	Description
candidate_id	string	☒ Yes	The CAND_XXXXXXX ID from candidates.jsonl
rank	int (1-100)	☒ Yes	The rank position. Must use each integer 1 through 100 exactly once.
score	float	☒ Yes	Your model's score for this candidate. Should be monotonically non-

			increasing as rank increases.
reasoning	string	⚠ Optional but strongly recommended	A 1-2 sentence justification explaining why this candidate is at this rank. Used at Stage 4 (manual review) to evaluate top submissions.

Example

```
candidate_id,rank,score,reasoning
CAND_0042871,1,0.987,"Senior AI Engineer with 7 years building RAG systems at product
companies; strong recent engagement and Bangalore-based."
CAND_0019884,2,0.973,"6 years applied ML; previously shipped vector search at scale;
matches the 'product over research' profile in the JD."
CAND_0091235,3,0.962,"Strong NLP + retrieval background; some concern on notice period
(120 days) but otherwise strong fit."
...
CAND_0007729,100,0.412,"Adjacent skills only – likely below cutoff but included as final
filler given experience and engagement signals."
```

3. Rules

Format

- Exactly 100 rows of data (plus 1 header row).
- Each rank (1 through 100) appears exactly once.
- Each candidate_id appears exactly once.
- Every candidate_id must exist in the released candidates.jsonl.
- score is non-increasing with rank — i.e., score at rank 1 $\geq$ score at rank 2 $\geq$... $\geq$ score at rank 100. Ties are allowed.
- If two candidates have the same score, you must still assign unique ranks. Break score ties deterministically using a secondary signal from your model, or by candidate_id ascending.

Compute constraints

Your code that produces the submission must satisfy the following constraints:

Constraint	Limit
------------	-------

Total runtime	≤ 5 minutes wall-clock
Memory	≤ 16 GB RAM
Compute	CPU only — no GPU during ranking
Network	Off — your ranking code must not make external API calls (no OpenAI, Anthropic, Cohere, Gemini, or any hosted LLM service)
Disk	≤ 5 GB intermediate state

Why these constraints? This is a real-world recruiting system, not a benchmark. A system that calls GPT-4 or Claude per candidate cannot scale to a 200K candidate pool in production. We want systems that have thought about latency-quality tradeoffs.

In practice, running an LLM call for each of 100,000 candidates will not fit the 5-minute CPU budget, even if the model runs locally. Plan for a small ranker over precomputed features, indexes, or compact local models.

You CANNOT, during the ranking step:

- Call hosted LLM APIs.
- Use GPUs.
- Exceed the runtime/memory limits.

Enforcement. At Stage 3, top-N submissions must provide their full code repository. Your ranking step will be reproduced inside a sandboxed Docker container matching these constraints exactly. If your submission cannot be reproduced within these limits, it is disqualified at Stage 3, regardless of your composite score. Make sure your code runs locally on a 16 GB CPU-only machine within 5 minutes before you submit.

Three-submission cap

You may make at most 3 submissions total during the competition window. Your final entry is your last valid submission. Earlier submissions are not preserved.

We've kept this number low intentionally — without a live leaderboard, multiple submissions have limited value, and a low cap reduces gaming.

Reasoning column

The reasoning column is optional but heavily recommended. Top N submissions are advanced to Stage 4 (manual review) where reasoning quality is part of the evaluation.

At Stage 4, we sample 10 random rows from your submission and check each reasoning entry against the following:

Check	What we're looking for
Specific facts	Does the reasoning reference specific facts from the candidate's profile (years of experience, current title, named skills, signal values)?
JD connection	Does the reasoning connect to specific JD requirements, not just generic praise?
Honest concerns	Where the candidate has obvious gaps or concerns, does the reasoning acknowledge them?
No hallucination	Does every claim in the reasoning correspond to something actually in the candidate's profile? Skills, employers, or experience that don't exist in the profile are red flags.
Variation	Are the 10 sampled reasonings substantively different from each other (not templated)?
Rank consistency	Does the reasoning's tone match the rank? A rank-5 candidate with critical reasoning, or a rank-95 candidate with glowing reasoning, indicates the reasoning was generated independently of the ranking.

What's penalized:

- Empty reasoning
- All-identical reasoning strings
- Templated reasoning that just inserts the candidate's name
- Reasoning that mentions skills not in the candidate's profile (hallucination)
- Reasoning that contradicts the rank

Plain-language reasoning that demonstrates you actually understood the candidate's profile will rank highly here. Don't try to be impressive; try to be specific and honest.

4. How submissions are scored

Metrics

Your top-100 ranking is scored against the hidden ground truth using these metrics:

Metric	Weight	What it measures
NDCG@10	0.50	Quality of your top-10 picks
NDCG@50	0.30	Quality of your top-50 picks
MAP (Mean Avg Precision)	0.15	Precision across all relevance levels
P@10	0.05	Fraction of top-10 that are "relevant" (tier 3+)

Final composite

Final composite = $0.50 \times \text{NDCG@10} + 0.30 \times \text{NDCG@50} + 0.15 \times \text{MAP} + 0.05 \times \text{P@10}$

Scoring happens once, after submissions close. There is no public partition, no live leaderboard, and no per-submission feedback during the competition. Your score is computed against the full hidden ground truth and is revealed only when final results are announced.

Tiebreaks

If two submissions have identical composites:

1. Higher P@5 wins.
2. Higher P@10 wins.
3. Earlier submission timestamp wins.

5. Evaluation pipeline (stages)

Your submission flows through these stages:

Stage	What happens	What gets you eliminated
1. Format validation	Auto-validator runs on every submission	Any spec violation in section 3
2. Scoring	Composite computed once on the full hidden ground truth, after submissions close	Final score below cutoff for advancement to Stage 3
3. Code reproduction + honeypot check	Top-N submissions: full code repo requested. Ranking step reproduced in sandboxed environment (5min, 16GB, no	Cannot reproduce within compute limits; honeypot rate >10% in top 100; missing or

	GPU, no network). Honeypot rate computed.	fabricated code repo
4. Manual review	Reasoning quality (6 checks above). Methodology coherence. Git history authenticity (real iteration vs single dump). Code quality.	Failed reasoning checks; flat git history with no iteration; codebase consists entirely of LLM API calls
5. Defend-your-work interview	Top X finalists: 30-minute video call with Redrob engineering. Walk through architecture, defend design choices, demonstrate familiarity with your own code.	Cannot explain architecture; contradicts submitted code; clearly didn't build it

Note on AI tool usage: You are allowed to use AI tools (Claude, GPT-4, etc.) as part of your development workflow. We expect many participants will. The evaluation is designed so that AI-assisted submissions where the human did real engineering work will succeed, while submissions that are mostly LLM output with minimal human engineering will fail at Stages 3-5. The compute constraint, code repo check, and defend-your-work interview together filter for genuine engineering, not for absence of AI use.

6. Common rejections (we see these every hackathon)

- 99 rows or 101 rows instead of exactly 100.
- Ranks starting at 0 instead of 1.
- Duplicate candidate_ids.
- candidate_id typos that don't exist in candidates.jsonl.
- All scores set to the same value (model isn't differentiating).
- Scores increasing as rank increases (rank 1 has lowest score).
- Submission file submitted as .xlsx or .json instead of .csv.

Double-check these locally before uploading — the server-side auto-validator rejects on any of them.

7. Honeypot warning

The dataset contains a small number (~80) of honeypot candidates with subtly impossible profiles (e.g., 8 years of experience at a company founded 3 years ago; "expert" proficiency in 10 skills with 0 years used). These are forced to relevance tier 0 in the ground truth.

If your submission ranks honeypots in the top 10, this is a strong signal that your system isn't reading profiles — it's just doing keyword embedding. We use the honeypot rate as a Stage 3 filter: submissions with honeypot rate > 10% in top 100 are disqualified.

You can identify honeypots through careful profile inspection. We expect a good ranking system to naturally avoid them; you don't need to special-case them.

8. Leaderboard policy

The leaderboard is hidden during the competition. You will not see your score until final results are announced. We strongly recommend you validate your approach locally using methodology and reasoning, not by submitting many variations.

9. Sample submission

A sample submission CSV that matches this spec is included in your hackathon bundle as `sample_submission.csv`. It is not a high-quality ranking — it's only a format reference.

10. What you submit (full picture)

Your submission consists of three parts, all required:

10.1 The CSV file

The top-100 ranking, as specified in Sections 2 and 3.

10.2 Portal metadata

Collected at upload time via the submission form. Have these ready before you start the upload:

Field	Required?	Notes
Team name	☒ Yes	Used in leaderboard and result announcements
Primary contact name	☒ Yes	One person to act as your team's point of contact
Primary contact email	☒ Yes	Used for all organizer communication
Primary contact phone	☒ Yes	Used for top-N / top-X

		communication
GitHub repository URL	✓ Yes	Must be reachable. Private repos OK if you can grant access to organizers at Stage 3. Format: https://github.com/USERNAME/REPO
Sandbox / demo link	✓ Yes	A working hosted environment where your ranking system can be run. See Section 10.5 below.
AI tools declared	✓ Yes	Multi-select: Claude / ChatGPT / Copilot / Cursor / Gemini / Other / None. Honest declaration, not penalized.
Compute environment summary	✓ Yes	One line describing where you ran your code (e.g., "MacBook Pro M2, 16GB RAM, Python 3.11")
Team member list	✓ Yes	Name + email for each member
Methodology summary	Optional	≤200 words explaining your approach. Strongly recommended — helps at Stage 4 review.

10.3 Code repository

Your GitHub repo should include:

- A clear README.md with setup instructions and exact commands to reproduce your submission CSV
- The full source code that produced the CSV (no hidden steps, no manual edits)
- Any pre-computed artifacts your code depends on (embeddings, indexes, model weights), or a script that produces them
- A requirements.txt, pyproject.toml, or equivalent specifying all dependencies and versions
- A submission_metadata.yaml at the repo root mirroring your portal metadata (use the template provided in the hackathon bundle as submission_metadata_template.yaml)

For Stage 3 code reproduction, your README must indicate a single command that produces the submission CSV from the candidates file. For example:

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

If your system requires pre-computation (e.g., generating embeddings), document this clearly — pre-computation may exceed the 5-minute window, but the ranking step that produces the CSV must complete within it.

10.4 AI tools declaration — what it means

The hackathon permits AI tool use. We've designed the evaluation pipeline so that AI-assisted submissions where the human did real engineering work will succeed, while AI-only submissions (paste-and-pray) will fail at Stage 3 (compute reproduction), Stage 4 (no real code repo), or Stage 5 (cannot defend the work).

The declaration is for transparency, not filtering. Be honest. If your interview answers contradict your declaration, that's a much stronger negative signal than the AI use itself.

10.5 Sandbox / demo link requirement

A sandbox is a hosted environment where organizers (and you) can verify your ranking system runs reproducibly. Acceptable sandbox platforms include:

- HuggingFace Spaces (free tier is fine)
- Streamlit Cloud (free tier is fine)
- Replit (public repl)
- Google Colab (with link to a notebook that runs end-to-end)
- A docker pull + docker run link to a public registry image
- A binder link for a runnable Jupyter notebook

Your sandbox needs to:

4. Accept a small candidate sample (≤ 100 candidates) as input — either via upload or pre-loaded
5. Run your ranking system end-to-end and produce a ranked CSV
6. Complete within the compute budget (≤ 5 min on CPU)

It does not need to handle the full 100K pool — small-sample reproducibility is what we're checking. The full reproduction at Stage 3 happens in our own sandbox.

Why it's mandatory: at Stage 3 we will reproduce your full ranking step from your GitHub repo. The sandbox is a faster, lower-stakes sanity check that lets us (and you) verify the code runs at all before we invest in full reproduction. Submissions without a working sandbox link are flagged at Stage 1.

If you have a strong reason a hosted sandbox is impractical for your approach, you can submit a self-contained docker run recipe in your GitHub README instead — but the dockerfile must build and run unmodified.